

GitHub Governance and Operational Policy

AI INSTITUTE AFRICA — DRAFT FRAMEWORK

Document Owner:	Software Development Standards Officer	Effective Date:	Pending Review
Status:	DRAFT - UNAPPROVED	Classification:	Internal Operational Use
Reference ID:	AIIA-GOV-GIT-2026		

DOCUMENT HISTORY & CONTROL

Version	Release Date	Summary of Changes / Basis	Author	Approved By
1.2	July 01, 2026	Updated metadata attribution to record explicitly that the policy text was prepared by Wills.	Wills	Pending Review
1.1	July 01, 2026	Revised framework update to formally include Section 2 (Definition of Terms) in compliance with standard software engineering directives.	Standards Office	Pending Review

1. Purpose and Scope

This policy defines the governance infrastructure, administrative mandates, and code modification workflows for utilizing GitHub within AI Institute Africa (AIIA). Version control software serves as the foundation for institutional codebase management, change logging, and distributed collaboration. This framework establishes uniform operational bounds to ensure code transparency, rigid configuration controls, and institutional security across all software development lifecycles.

The scope of this policy spans all engineering personnel, code contributors, temporary consultants, and third-party contractors executing work within or on behalf of AIIA. It covers all software repositories, tracking environments, and configuration assets housed under the official corporate cloud organizational profile.

2. Definition of Terms

The following technical terms used throughout this document are defined in alignment with baseline Git operations, GitHub cloud management mechanisms, and the foundational framework:

- **Git:** A distributed version control system that operates without a central service, storing complete snapshots of files across multiple revisions to enable unalterable audit trails and change tracing.
- **GitHub:** A centralized cloud hosting platform providing integrated management interfaces for Git repositories, encompassing issue trackers, user roles, continuous integration hooks, and organizational groups.
- **Organization:** A centralized, uniformly named namespace footprint on GitHub designed to manage teams, unify access controls, and systematically isolate repository groups under formal technical ownership.
- **Repository (Repo):** An isolated digital storage space hosting a specific application's source code, tracking lines, configuration properties, issue queues, and development documentation.

- **Branch:** A distinct tracking track within a repository that allows specific modifications to be isolated from other workflows before final integration into primary streams.
- **Pull Request (PR):** A formal GitHub mechanism proposing code changes from a secondary feature workspace into a protected track, acting as an unalterable review log for validation signatures.
- **Issue Tracker:** An integrated system within a repository utilized to document bugs, log enhancements, categorize tasks via labels, and record the final historical consensus of decisions.
- **Continuous Integration (CI):** Automated third-party service hooks that systematically build, compile, and validate code submissions for syntactic, formatting, or security errors prior to merging.
- **Multi-Factor Authentication (MFA):** A strict identity verification security constraint requiring users to provide two or more distinct verification proofs prior to gaining organization access.

3. Administrative Policies and Organizational Architecture

In alignment with established architectural best practices and distributed version control standards, all code developments must conform to a centralized governance perimeter:

- **Centralized Corporate Organization:** Ad-hoc or personal repositories are strictly prohibited for corporate-backed intellectual assets. All code projects must live within a dedicated, authorized GitHub Organization namespace. Organizations should utilize consistent naming conventions to guarantee clear discoverability and formal management.
- **Administrative Multiplexing (Owner Access):** To eliminate dangerous operational single points of failure, each organization must maintain a minimum of two active, independent identities with owner permissions. Administrative ownership must include executive technical leadership (e.g., Chief Technology Officer or Chief Information Officer) alongside designated software compliance and security officers to prevent service deadlocks.
- **Role-Based Access Control (RBAC):** Access rights inside the code repository space are strictly limited using the principal of least privilege, grouped into explicit GitHub Teams:
 - *Admin Access:* Reserved for repository owners and technical chairs to regulate system parameters, construct webhooks, and configure branch protection patterns.
 - *Write Access:* Formally allocated to primary software developers, systems engineers, and document editors to push changes to feature workspaces.
 - *Read Access:* Configured for quality assurance auditors, casual reviewers, and data consumers to track issues and pull requests without alteration capabilities.

4. Repository Integrity and Perimeter Transparency

To avoid overhead, reduce operational noise, and contain security perimeter vulnerabilities, repository architectures must adhere to strict logical isolation boundaries:

- **Granular Isolation Matrix:** Distinct software frameworks, architectural microservices, and specialized modules must be maintained in independent repositories. Combining unrelated application components or sprawling project clusters within single unsegmented repositories introduces permission bleeding and is forbidden.

- **Default Perimeter Configuration:** Repositories housing proprietary code, core infrastructure automation, enterprise data engines, or production algorithms must maintain a strict Private visibility setting. Public settings are permitted only for specialized open-source tools or public specifications formally vetted and approved by the engineering board.
- **Onboarding Documentation Requirements:** Every active repository instance must natively include baseline tracking records:
 - *CONTRIBUTING.md*: Serving as the definitive operational manual for the repository, detailing specific local branching paradigms, testing conditions, and integration workflows.
 - *README.md*: Delivering a clear technical summary of the codebase, system requirements, and a mandatory routing link straight to the contribution manual.

5. Branching Topology and Environment Isolation

To support corporate Software Development Lifecycle (SDLC) mandates enforcing rigid logical isolation between Development, Staging, and Production tiers, the following branching architecture is required across all repositories:

- **Production Tracking Track (main or production):** Mirrors the current, fully verified state of live production infrastructure. Direct manual commits or unvetted push streams into this line are strictly blocked by system configurations.
- **Staging and Testing Track (staging or testing):** Used exclusively for system integration testing, quality assurance runs, performance analysis, and pre-release security validation.
- **Feature Workspaces (development or ephemeral local feature branches):** Open collaborative zones used by engineers for isolated development tasks, code prototyping, and individual feature construction.

Both production and staging tracks must be fortified via automated branch protection rules. These protections mandate linear history layouts, block non-linear squashes that disrupt history tracking, require all automated Continuous Integration (CI) status checks to complete successfully, and require at least one explicit code review sign-off from an independent editor before code integration can proceed.

6. Contribution Models and Issue Management Workflows

Every alteration to an active software asset must pass sequentially through an unalterable change verification lifecycle. Ad-hoc changes bypassing logging mechanisms are treated as critical compliance violations.

6.1 Regulated Issue Tracking Modes

The repository issue tracker serves as the primary system of record for bug logging, platform enhancements, and technical debt tracking. Tasks must be classified at creation using an explicit dual-label mechanism to govern discussion venues:

- **design Label (Substantive):** Applied to tasks modifying core application logic, data schemas, communication interfaces, or baseline system interoperability. For mature components, design decisions must be elevated to official technical mailing lists or architectural review boards to ensure comprehensive consensus tracking. design issues can only be closed once an authorized change token is logged.

- **editorial Label (Minor):** Applied to code comment revisions, typographical corrections, documentation adjustments, or visual style layout fixes. Resolution of editorial concerns is left to the direct discretion of active repository editors.

6.2 Pull Request Execution and Segregation of Duties

All substantive code modifications must execute via a Pull Request (PR) rather than a direct push. PR workflows generate structured artifacts capturing code changes, testing validations, and developer rationales. Technical discussions regarding architectural validity should be maintained in the main PR review thread rather than inline code review comments to ensure structural history is not lost during subsequent code updates.

To enforce absolute segregation of duties, the final merge of a validated pull request into a protected deployment track (such as staging or production) must be performed by an independent deployment manager or repository chair who holds no authoring signature or direct stake in the underlying patch set.

7. Continuous Integration (CI) Standards

Automated continuous integration hooks must link directly to the version control lifecycle to inspect code health dynamically prior to code integration:

- **Syntactic and Structural Health Check:** Automated workflows must parse each submission for syntax compliance, runtime compilation integrity, and formatting linter alignments. Failure to compile instantly flags the PR as unmergeable.
- **Vulnerability Mitigation Profiling:** Code differences must be scanned automatically for known architectural flaws, third-party package vulnerabilities, and security risks mapped against standard security frameworks like the OWASP Top 10 vulnerabilities (including injection vulnerabilities and broken access controls).

8. Security and Institutional Business Continuity

To ensure high operational resilience and protect institutional intellectual capital against compromise or external platform failures, the following security baselines are strictly enforced:

- **Identity Verification:** Multi-Factor Authentication (MFA) is strictly mandatory for all credentials holding membership inside the official corporate GitHub organization footprint.
- **Credential Sanitization:** Hardcoding secret assets, API authorization vectors, private keys, or configuration strings inside source control lines is completely forbidden. Repositories must use automated secret scanning tools to block any commits containing exposed credentials.
- **Metadata Backups and Platform Resilience:** While git repositories are distributed by design, associated operational metadata (such as issue tracking history, pull request discussions, architecture reviews, and configuration settings) are highly vulnerable to localized cloud service interruptions. Automated operational tasks must execute periodically to extract and replicate all repository code and tracking metadata to separate, secure, off-site corporate storage environments.

9. Compliance and Policy Enforcement

Full compliance with this governance structure is mandatory to ensure the stability, safety, and auditability of AIIA information systems. Circumventing automated protection controls, deliberately masking proprietary assets via incorrect visibility settings, or pushing code additions outside authorized change workflows introduces significant institutional risk. Violations will trigger immediate internal investigations and disciplinary enforcement protocols, up to and including immediate termination of engagement contracts.